



DEVELOPER COMMUNITY LEARNING MATERIAL

HTML

Interview & Placement Master Roadmap

A comprehensive guide for college students and freshers to master HTML structure, modern web semantics, and essential interview concepts.

Learning Roadmap

The complete path to HTML mastery for freshers.

- 01 Fundamentals:** What is HTML, Structure, Tags vs Elements, Attributes, Block vs Inline
- 02 Text & Links:** Headings, Paragraphs, Anchors, URLs, Targets
- 03 Lists, Images & Tables:** Semantics of tabular data, figure/figcaption
- 04 Forms:** Inputs, Validation, GET vs POST, Accessibility
- 05 Semantic HTML:** header, nav, main, section vs div, SEO
- 06 Attributes & Accessibility:** Global attrs, ARIA, Web Accessibility basics
- 07 Head & Integration:** Meta tags, Viewport, script async vs defer
- 08 DOM & Browser Basics:** DOM Tree, Nodes, DevTools
- 09 Practical HTML:** Profile pages, Registration, Portfolios
- 10 Interview Preparation:** Top fresher placement questions categorized

HTML Fundamentals

💡 What is HTML?

HTML stands for **HyperText Markup Language**. It is not a programming language; it is a markup language used to structure web pages and their content (like paragraphs, lists, and images).

The HTML Document Structure

Every standard HTML page follows a specific skeleton. Interviewers often ask you to write this basic structure on a whiteboard.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My Page Title</title>
  </head>
  <body>
    <!-- Content goes here -->
  </body>
</html>
```

Component	Why does it exist? (Purpose)
<code><!DOCTYPE html></code>	Tells the browser this is an HTML5 document. Without it, browsers may enter "quirks mode" and render pages incorrectly.
<code><html></code>	The root element containing all other HTML elements.
<code><head></code>	Contains meta-information, title, and links to CSS/JS. It is not visible on the page.
<code><body></code>	Contains all the visible content (text, images, forms) displayed in the browser.

Tags vs Elements

These terms are often confused by beginners.

- **Tag:** The keywords enclosed in angle brackets (e.g., `<p>` is an opening tag, `</p>` is a closing tag).
- **Element:** The complete block, including the opening tag, the content, and the closing tag.

🎯 Placement Focus

Q: Do all HTML tags have a closing tag?

A: No. Some elements are "void elements" (or self-closing tags) because they don't contain text or other elements. Examples: `<img>`, `<br>`, `<hr>`, `<input>`.

Attributes

Attributes provide additional information about an element. They are always placed in the **opening tag** and usually come in name/value pairs: `name="value"`.

```
<a href="https://godix.in" class="primary-link">Visit GODIX</a>
```

In the above example, `href` and `class` are attributes.

Comments in HTML

Comments are used to explain code and are ignored by the browser. `<!-- This is a comment -->`

Block vs Inline Elements

This is one of the most fundamental concepts in HTML and CSS layout.

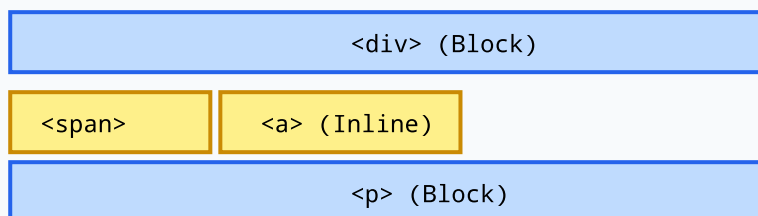
Block-level Elements

- Always start on a new line.
- Take up the **full width** available.
- Can contain inline elements and other block elements.
- Examples: `<div>`, `<p>`, `<h1>`, `<section>`

Inline Elements

- Do not start on a new line.
- Take up **only as much width as necessary**.
- Cannot contain block elements.
- Examples: `<span>`, `<a>`, `<strong>`, `<img>`

Browser Window Width



Interview Question: div vs span?

Answer: `<div>` is a generic block-level container used to group block-elements for layout. `<span>` is a generic inline container used to group text or inline-elements for styling without breaking the text flow to a new line.

Text & Links

Text Formatting

Tag	Purpose	Visual Result
<code><h1></code> to <code><h6></code>	Headings. H1 is most important (SEO), H6 least.	Bold, varying sizes.
<code><p></code>	Paragraphs. Blocks of text.	Standard text block.
<code></code>	Defines important text (semantic).	Bold text
<code></code>	Defines emphasized text (semantic).	<i>Italic text</i>
<code>
</code> / <code><hr></code>	Line break / Thematic break (Horizontal line).	Line break / Horizontal line.

⚠ Common Mistake

Do not use `<b>` and `<i>` tags for modern web development. Use `<strong>` and `<em>` because they provide **semantic meaning** for screen readers (accessibility), whereas ``b`` and ``i`` are purely visual.

Links: The `<a>` Tag

The anchor tag creates hyperlinks. The most important attribute is `href` (Hypertext REference), which indicates the link's destination.

```
<a href="https://google.com" target="_blank">Search Google</a>
```

Absolute vs Relative URLs

Absolute URL

Points to an external website. It includes the full protocol (http/https) and domain name.

```
href="https://api.github.com/users"
```

Relative URL

Points to a file within the *same* website. It does not include the domain name.

```
href="/images/profile.jpg"
```

```
href="../about.html"
```

Important Link Attributes

- `target="_blank"` : Opens the linked document in a new tab/window.
- Fragment Links (id)**: You can link to a specific part of a page by assigning an `id` to an element, and using `#id` in the href.

```
<h2 id="contact">Contact Us</h2>
<!-- Clicking this link jumps to the heading above -->
<a href="#contact">Go to Contact Section</a>
```

Lists & Images

Lists (ul, ol, li)

HTML provides two main ways to structure lists. Both use `<li>` (List Item) inside them.

Unordered List `<ul>`

Used when order doesn't matter (bullet points).

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
</ul>
```

Ordered List `<ol>`

Used when order matters (numbered lists).

```
<ol>
  <li>First Step</li>
  <li>Second Step</li>
</ol>
```

Nested Lists: You can place a list inside another list item. This is commonly used for dropdown navigation menus in real software development.

Images (img, figure, figcaption)

The `<img>` tag is used to embed images. It is an inline, empty tag (no closing tag).

Attribute	Importance & Usage
<code>src</code>	(Source) Specifies the path to the image file. Mandatory.
<code>alt</code>	(Alternate text) Shown if the image fails to load. Crucial for Screen Readers (Accessibility) and SEO. Always include it.

Figure and Figcaption

When you need to provide a caption for an image, use the semantic `<figure>` element instead of just a `<div>`.

```
<figure>
  
  <figcaption>Fig 1: Client-Server Architecture.</figcaption>
</figure>
```

🎯 Placement Focus

Interviewers often check your code to see if you included the `alt` attribute on images. Leaving it empty or omitting it shows a lack of awareness of Web Accessibility guidelines.

HTML Tables

Tables are used to display tabular data (grids of data). **Do not use tables for page layout** (use CSS Flexbox/Grid instead).

Table Structure

A standard, semantic HTML table is broken into three main sections: Header, Body, and Footer.

```
<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Rahul</td>
      <td>22</td>
    </tr>
  </tbody>
</table>
```

- `<table>` : Container for the table.
- `<tr>` : Table Row.
- `<th>` : Table Header (bold, centered default).
- `<td>` : Table Data (standard cell).
- `<thead>` : Semantic grouping for headers.
- `<tbody>` : Semantic grouping for main content.
- `<tfoot>` : Semantic grouping for summary/footer rows.

Rowspan and Colspan

These attributes allow a cell to span across multiple rows or columns, merging them.

colspan="2"

Data	Data
<td colspan="2">	

rowspan="2"

<td rowspan="2">	Data
	Data

Practice Task

Create a Student Grade table. Use `<thead>` for columns (Name, Math, Science, Total). Add 3 student rows in `<tbody>`. Use `<tfoot>` and a `colspan="3"` cell to write "Class Average", with the final average in the last cell.

Forms

Forms are how users send data to a server (e.g., logging in, uploading files, searching). They are heavily tested in interviews because they are critical to interactive web applications.

The `<form>` Element

The `<form>` tag acts as a container for all inputs. Its two most important attributes are `action` (where to send data) and `method` (how to send data).

🔗 Interview Question: GET vs POST Method

Feature	GET Method	POST Method
Visibility	Data is visible in the URL (e.g., <code>?name=rahul&age=22</code>).	Data is hidden in the HTTP request body.
Security	Insecure. Never use for passwords.	More secure. Always use for sensitive data.
Limits	URL length is limited (~2000 chars).	No practical size limit (good for files).
Use Case	Fetching data, Search queries, Filters.	Submitting data, Login, Uploads.

The `<input>` Element

The most versatile form element. Its appearance changes based on the `type` attribute.

`type="text"` Standard text box.

`type="password"` Hides characters.

`type="email"` Validates @ symbol.

`type="number"` Numeric only.

`type="date"` Calendar picker.

`type="file"` File upload.

The Importance of `<label>`

Labels describe inputs. You link a label to an input using the `for` attribute matching the input's `id`. This makes the form accessible and allows users to click the label to focus the input.

```
<label for="username">Username:</label>
<input type="text" id="username" name="user">
```

Radio Buttons vs Checkboxes

Radio `type="radio"`

Allows selecting **only ONE** option from a group. All radios in the group must share the same `name` attribute.

```
<input type="radio" name="gender" value="m"> Male
<input type="radio" name="gender" value="f"> Female
```

Checkbox `type="checkbox"`

Allows selecting **ZERO OR MORE** options. Independent state.

```
<input type="checkbox" name="hobbies"> Coding
<input type="checkbox" name="hobbies"> Reading
```

Other Input Types: Select & Textarea

- `<select>` and `<option>` : Creates a dropdown list.
- `<textarea>` : Creates a multi-line text input (e.g., for comments). *Note: Textarea is not a void element, it has a closing tag.*

Important Form Attributes

💡 name vs id

`id` is used by CSS for styling and JS/Labels for selecting the exact element. `name` is the variable name used by the backend server to identify the data when the form is submitted.

🎯 Interview Question: disabled vs readonly?

readonly: User cannot edit the value, but they can select/copy it. The data **is sent** when the form submits.

disabled: User cannot edit, select, or focus it. The element is greyed out. The data **is NOT sent** when the form submits.

Client-Side Validation

HTML5 provides built-in validation attributes to prevent users from submitting bad data without needing JavaScript.

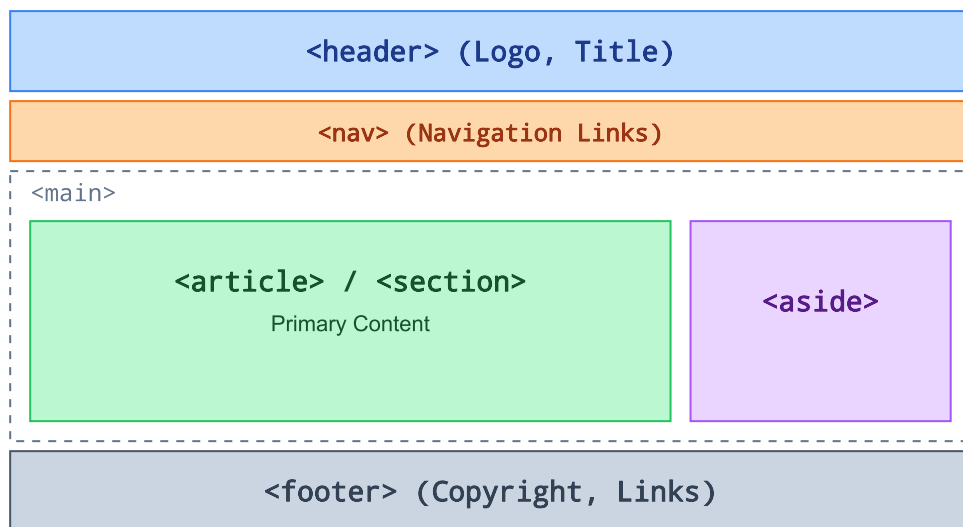
- `required` : Input cannot be empty.
- `min` / `max` : Number range limits.
- `minlength` / `maxlength` : Text length limits.
- `pattern` : Enforces a Regex pattern (e.g., `pattern="[A-Za-z]{3}"`).

Semantic HTML

💡 What is Semantic HTML?

Semantic HTML means using HTML tags that clearly describe their meaning to both the browser and the developer. Instead of using `<div>` for everything, you use tags like `<article>` or `<footer>` to define the architecture of the page.

Typical Semantic Page Layout



🔗 Interview Question: div vs section vs article?

- `<div>` : Has no semantic meaning. Use only for styling/CSS grouping.
- `<section>` : A thematic grouping of content, usually containing a heading (e.g., "Contact Us" section).
- `<article>` : A completely independent, self-contained piece of content that could be distributed on its own (e.g., a blog post or news article).

Why is Semantic HTML Important?

1. **SEO (Search Engine Optimization)**: Search engines prioritize content in `<main>`, `<h1>`, and `<article>` tags over generic divs.
2. **Accessibility**: Screen readers use semantic tags to navigate the page, letting blind users skip straight to the `<nav>` or `<main>`.
3. **Maintainability**: Code is much easier for other developers to read than a "soup" of divs.

Attributes & Accessibility

Global vs Boolean Attributes

Global Attributes can be used on *almost any* HTML element.

- **id** : A unique identifier for the element. (Max 1 per page).
- **class** : Used to assign multiple elements to a group for CSS styling.
- **title** : Displays a tooltip when hovering over the element.
- **lang** : Defines the language (e.g., `<html lang="en">`).

Boolean Attributes do not require a value. Their presence alone means "true".

- Examples: `disabled` , `checked` , `required` , `readonly` .
- Correct: `<input type="checkbox" checked>`

Data Attributes (`data-*`)

Used to store custom, invisible data on an element that can be easily accessed by JavaScript.

```
<button data-product-id="4521" data-price="99">Buy Now</button>
```

Web Accessibility (a11y)

Accessibility ensures that websites are usable by people with disabilities (vision, motor, cognitive).

Keyboard Navigation

Users must be able to navigate the site using only the `Tab` and `Enter` keys. Semantic interactive elements (`<a>` , `<button>` , `<input>`) are keyboard accessible by default.

Screen Readers

Software that reads the screen out loud. Always provide `alt` text for images and use `<label>` for forms.

Basic ARIA Concepts

ARIA (Accessible Rich Internet Applications) attributes help provide semantic meaning when HTML tags aren't enough (often used in complex JavaScript widgets).

- `aria-label="text"` : Provides an invisible label for a screen reader when a visible label isn't present (e.g., an icon-only "X" close button).
- `role="button"` : Tells the screen reader an element behaves like a button, even if it's coded as a `<div>` (though you should just use a real `<button>`).

HTML Head & Integration

The `<head>` Tag

The head contains meta-information that configures how the page behaves.

- `<meta charset="UTF-8">` : Ensures symbols/emojis render correctly.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">` : **Crucial for Mobile Responsiveness**. Without this, websites look tiny on mobile phones.
- `<title>` : Text shown in the browser tab.
- `<link rel="icon" href="favicon.ico">` : The small icon in the browser tab.

Integration: CSS and JavaScript

Web development uses a separation of concerns: HTML (Structure), CSS (Style), JS (Logic).

Adding CSS: Use the `<link>` tag inside the `<head>` .

```
<link rel="stylesheet" href="styles.css">
```

Adding JS: Use the `<script>` tag, usually placed just before the closing `</body>` tag, OR in the head using `defer` .

```
<script src="app.js" defer></script>
```

🔗 Deep Dive: script async vs defer

When the browser parses HTML and hits a normal `<script>` , it pauses HTML parsing to download and execute the script. This blocks the page from loading. We fix this with `async` or `defer` .

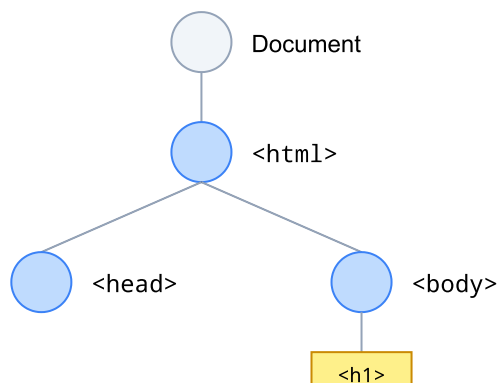
Normal Script	Blocks HTML parsing while downloading and executing.
async	Downloads in the background, but pauses HTML parsing when it's ready to execute. Execution order is NOT guaranteed. Good for analytics scripts.
defer	Downloads in the background, but waits to execute until the HTML parsing is fully finished . Execution order is guaranteed. Best for DOM manipulation scripts.

DOM & Browser Basics

💡 What is the DOM?

DOM stands for **Document Object Model**. When a browser loads an HTML file, it converts the tags into a tree-like object structure in memory. JavaScript interacts with this DOM tree, not the raw text file.

The DOM Tree



Element Nodes and JavaScript

Because the HTML becomes a DOM tree of "Nodes", JavaScript can select them and modify their content, styles, or attributes.

```
// 1. Selecting an element by its ID
const heading = document.getElementById("main-title");

// 2. Changing the text content
heading.innerText = "Welcome to GODIX!";

// 3. Changing styles directly
heading.style.color = "blue";
```

Browser DevTools (Elements Tab)

Every developer must master the browser's Developer Tools (Right Click -> Inspect, or F12).

- The **Elements tab** shows the live DOM tree. It is NOT exactly your source code; it is what the browser built (including JS changes).
- You can live-edit text, attributes, and CSS in this tab to experiment without refreshing.

⚠️ Common Mistake

Beginners often think "View Page Source" and "Inspect Element" are the same. "View Source" shows the raw HTML file from the server. "Inspect Element" shows the live, current DOM state which JavaScript might have already modified.

Practical HTML Architecture

In interviews and practical rounds, you are often asked to structure a page. Focus on proper semantic hierarchy rather than CSS.

Portfolio Website Structure

```
<header>
  <nav>
    <ul>
      <li><a href="#about">About</a>
    </li>
      <li><a href="#projects">Projects</a></li>
    </ul>
  </nav>
</header>

<main>
  <section id="about">
    <h1>John Doe</h1>
    <p>Frontend Developer</p>
  </section>

  <section id="projects">
    <h2>My Work</h2>
    <article>
      <h3>Project 1</h3>
      
    </article>
  </section>
</main>

<footer>© 2024 John Doe</footer>
```

Registration Form

```
<form action="/register"
method="POST">
  <fieldset>
    <legend>Account Details</legend>

    <div>
      <label for="email">Email:
    </label>
      <input type="email" id="email"
name="email" required>
    </div>

    <div>
      <label for="pwd">Password:
    </label>
      <input type="password"
id="pwd"
name="password"
minlength="8">
    </div>

    <button
type="submit">Register</button>
  </fieldset>
</form>
```

Mini Task: Job Application Page

Build a `job-app.html` page. Use a `<header>` for the company logo. Use a `<form>` with method POST. Include text inputs for Name/Email, a `<select>` for Role, a `<input type="file">` for Resume, and a Submit button. *Ensure every input has a linked label!*

Placement Interview Bank

Top questions asked during fresher technical screening rounds.

Q1: What is the purpose of the `alt` attribute on images?

A: It provides alternative text if the image fails to load. More importantly, it is read by screen readers for visually impaired users (accessibility) and is indexed by search engines (SEO).

Q2: Differentiate between `<script>` , `<script async>` , and `<script defer>` .

A: Standard script blocks HTML parsing. `async` downloads in the background and executes immediately when ready, halting HTML parsing (order not guaranteed). `defer` downloads in the background but waits until the HTML is fully parsed before executing (preserves order).

Q3: Why should we use Semantic HTML?

A: Semantic tags (like `<article>` , `<nav>` , `<header>`) give meaning to the web page structure. This improves SEO, makes the site accessible to screen readers, and makes the codebase easier for developers to read compared to using endless `<div>` tags.

Q4: What is the difference between Block and Inline elements?

A: Block elements (`div` , `p` , `h1`) start on a new line and take up 100% of the available width. Inline elements (`span` , `a` , `strong`) do not start on a new line and only take up as much width as their content requires.

Q5: Explain GET vs POST in form submission.

A: GET appends form data to the URL (visible, limited length, used for fetching data like searches). POST sends data inside the HTTP request body (hidden from URL, no length limit, used for sensitive data like passwords or large data like files).

Quick Revision Checklist Before Interviews

- Can you write the HTML5 boilerplate from memory?
- Do you know the difference between ID and Class?
- Can you explain the DOM tree?
- Do you understand form attributes (name vs id, disabled vs readonly)?